머신러닝

3주차. 데이터 시각화 / matplotlib

학습 목표

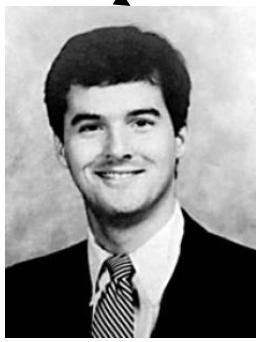
- **Matplotlib 소개**
- **Matplotlib 기본 기능 및 함수**
- **Matplotlib 그래프**
  - 막대그래프(Bar, Barh)
  - 산포도(Scatter)
  - 파이 차트(Pie Chart)
  - 히스토그램(Histogram)

- **Pandas, Numpy에서 Matplotlib 사용**

Matplotlib 소개

- **데이터를 그래프나 플롯(plot)으로 시각화(visualization)를 위한 파이썬의 대표적인 라이브러리**
  - 그래프나 그림을 그릴 수 있는 플롯팅(plotting) 라이브러리
  - **플롯(Plot)**: 그래픽으로 렌더링하여 화면으로 출력하기 전에 데이터를 가상의 공간에 미리 그려 놓는 것

- **존 D. 헌터(John D. Hunter)에 의해 개발**
  - 출판물에 필요한 수준의 그래프를 만드는데 초점
  - 주로 사용하는 MATLAB에 영감을 얻어 개발



matplotlib

Matplotlib 소개(계속)

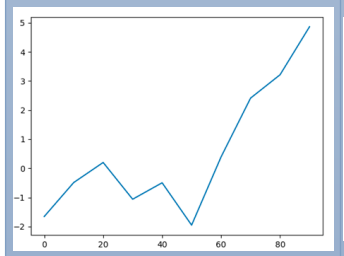
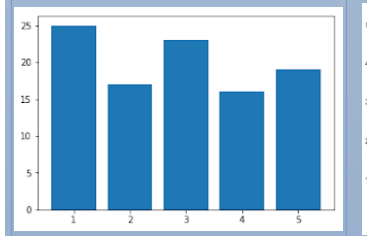
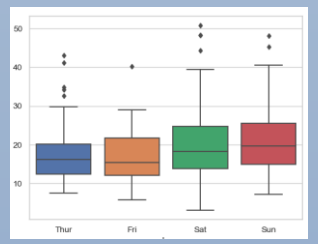
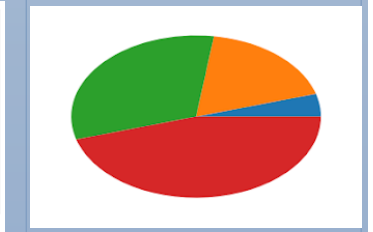
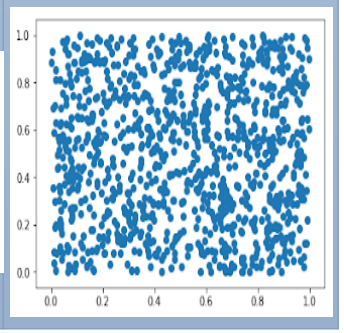
- **데이터 분석 이전에 데이터 이해를 위한 시각화 또는 분석 후에 결과를 시각화하기 위해서 사용**
  - 파이썬에서 Matplot의 기능을 인터렉티브하게 사용할 수 있음
  - Pyplot 모듈을 사용하여 데이터를 시각적으로 표현

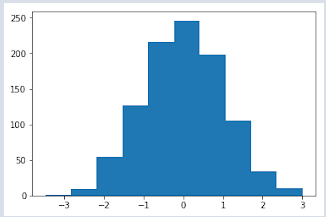
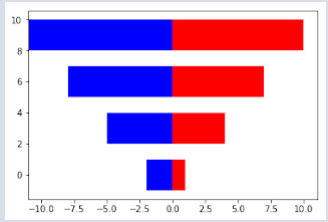
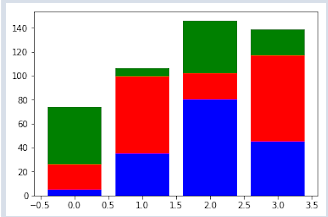
```
from matplotlib import pyplot as plt
or
import matplotlib.pyplot as plt
```

- **다양한 Matplotlib 샘플** https://matplotlib.org/gallery/index.html

Matplotlib 소개(계속)

- **Matplotlib 다양한 시간화 기능**

plot line	bar (막대그래프)	box	pie (pie 차트)	scatter (산포도)
				

histogram (히스토그램)	back-to-back	stack bar
		

Matplotlib 기본 기능 및 메소드 함수

- **Matplotlib 기본 메소드 함수**

함수	기능	사용방법
plot()	입력 데이터로 시각화(plot) 데이터 생성 x, y 입력 데이터 - 데이터 (리스트, 튜플) - pandas의 Series - numpy array	plt.plot(리스트) plt.plot(판다스_스리즈)
xlabel() ylabel()	x, y축 이름 출력	plt.xlabel ("문자열") plt.ylabel ("문자열")
title()	생성된 그래프의 제목 출력	plt.title ("문자열")
show()	생성된 시각화 데이터 출력	plt.show()
savefig()	생성한 시각화 이미지 저장 입력 데이터 - 저장할 이미지 파일명	plt.savefig("my_plot.png")

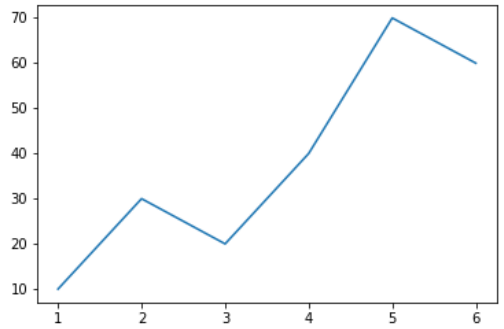
Matplotlib 기본 기능 및 메소드 함수(계속)

- 기본 그래프

```
from matplotlib import pyplot as plt

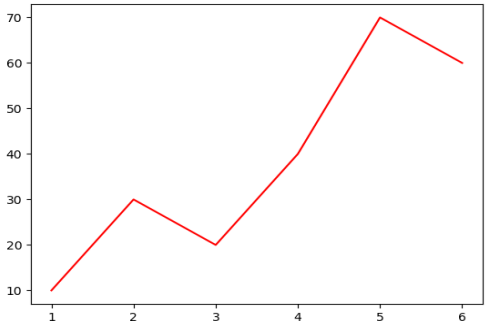
x = [1, 2, 3, 4, 5, 6]
y = [10, 30, 20, 40, 70, 60]

plt.figure()
plt.plot( x, y)            # plot  생성
plt.show()
```



- 색상 변경

```
plt.plot( x, y,  "r" )          # 색상 변경
plt.show()
```

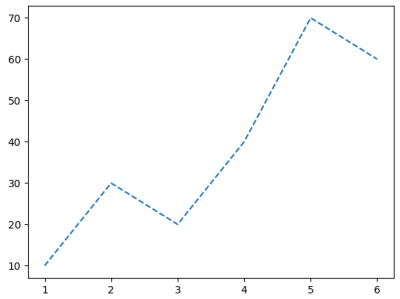


Character	Color
'b'	Blue
'g'	Green
'r'	Red
'c'	Cyan
'm'	Magenta
'y'	Yellow
'k'	Black
'w'	White

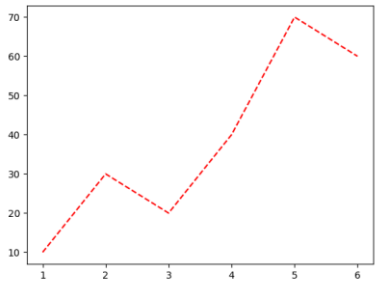
Matplotlib 기본 기능 및 메소드 함수(계속)

- 선 종류 변경

```
plt.plot( x, y, '--')          #선 모양 변경
plt.show()
```




```
plt.plot( x, y, '--r')         # 선 모양 + 색상 변경
plt.show()
```

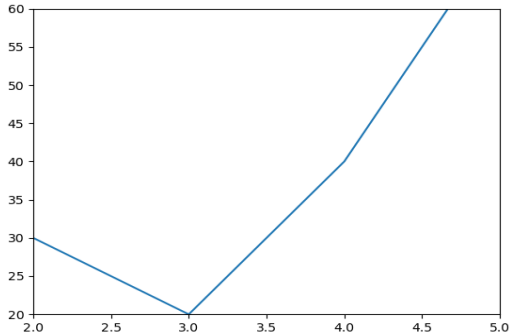


Character	Description
'-'	Solid line style
'--'	Dashed line style
'-.'	Dash-dot line style
':'	Dotted line style
'.'	Point marker
','	Pixel marker
'o'	Circle marker
'v'	Triangle_down marker
'^'	Triangle_up marker
'<'	Triangle_left marker
'>'	Triangle_right marker
'1'	Tri_down marker
'2'	Tri_up marker
'3'	Tri_left marker
'4'	Tri_right marker
's'	Square marker
'p'	Pentagon marker
'*'	Star marker
'h'	Hexagon1 marker
'H'	Hexagon2 marker
'+'	Plus marker
'x'	X marker
'D'	Diamond marker
'd'	Thin_diamond marker
'|'	Vline marker
'_'	Hline marker

Matplotlib 기본 기능 및 메소드 함수(계속)

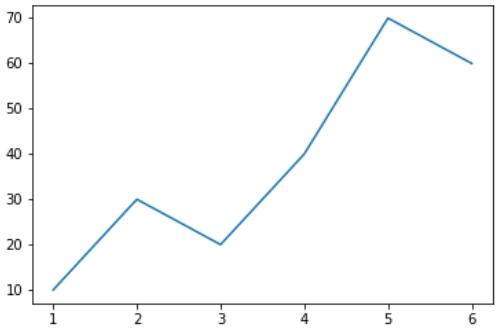
- 그래프 구간 조절 (plt.xlim(), plt.ylim())

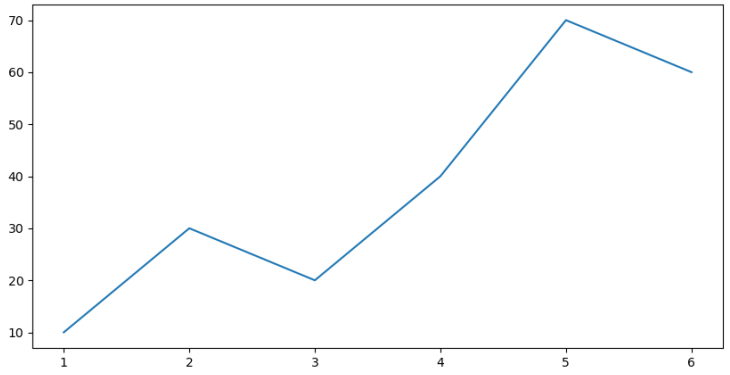
```
plt.xlim(2, 5)        # x축 구간 변경
plt.ylim(20, 60)      # y축 구간 변경
plt.plot( x, y)
plt.show()
```



- 이미지 사이즈 조절 (plt.figure(figsize = ())

```
plt.figure(figsize = (30, 5))
```





Matplotlib 기본 기능 및 메소드 함수(계속)

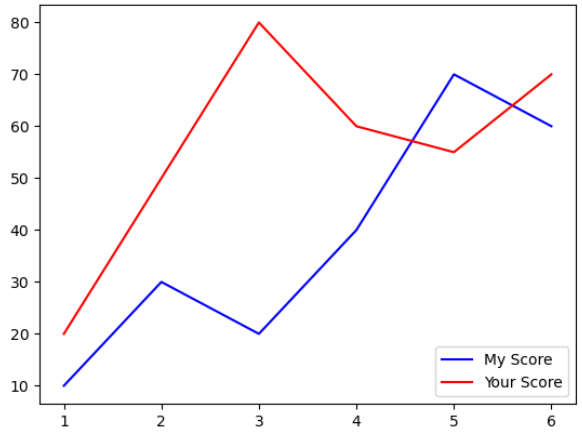
- 항목, 범주 표기: plt.legend()

```
from matplotlib import pyplot as plt

x = ["1", "2", "3", "4", "5", "6"]
my_y = [10, 30, 20, 40, 70, 60]
you_y = [20, 50, 80, 60, 55, 70]
plt.plot( x, my_y, 'b', label = "My Score")          # plot  생성
plt.plot( x, you_y, 'r', label = "Your Score")        # plot  생성

#plt.legend(loc = 4)
plt.legend(loc = 'lower right)

plt.show()
```



string	code	string	code	string	code
'upper left'	2	'upper center'	9	'upper right'	1
'center left'	6	'center'	10	'center right'	7
'lower left'	3	'lower center'	8	'lower right'	4
'best'	0			'right'	5

Matplotlib 기본 기능 및 메소드 함수(계속)

- 어노테이션(주석): plt.annotate()

```
plt.annotate("문자열",  xy = (), xytext =(), arrowprops = {        })
```

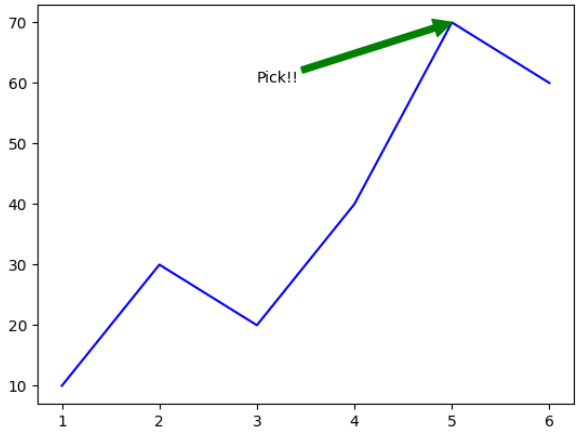
```
from matplotlib import pyplot as plt

x = [1, 2, 3, 4, 5, 6]
y = [10, 30, 20, 40, 70, 60]

plt.annotate('Pick !', xy=(5,70),  xytext=(3,60), arrowprops={'color':'green'})

plt.plot( x, y)            # plot  생성

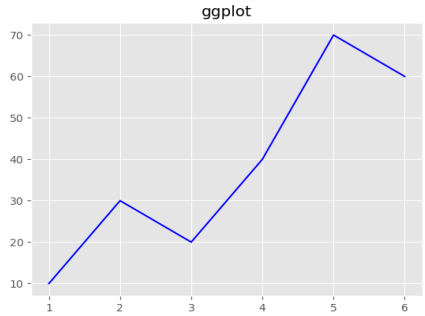
plt.show()
```



Matplotlib 기본 기능 및 메소드 함수(계속)

- 그래프 스타일 변경: plt.style.use()

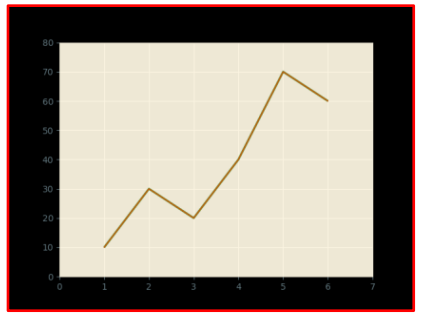
```
plt.style.use("ggplot")
plt.title("ggplot")
plt.show()
```

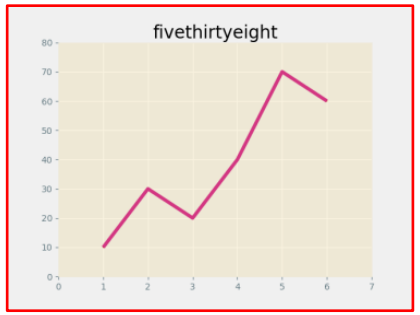


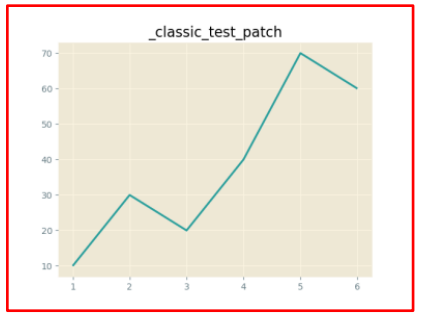
- 스타일 목록 확인(28종류): plt.style.available

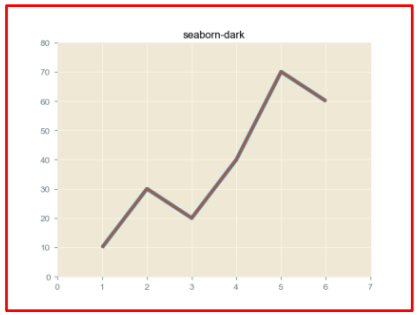
```
print(plt.style.available)
```

```
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeigh
t', 'ggplot', 'grayscale', 'seaborn-v0_8', 'seaborn-v0_8-bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark', 'seaborn-v0_8-dark-palette',
'seaborn-v0_8-darkgrid', 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-notebook', 'seaborn-v0_8-paper', 'seaborn-v0_8-pastel', 'sea
born-v0_8-poster', 'seaborn-v0_8-talk', 'seaborn-v0_8-ticks', 'seaborn-v0_8-white', 'seaborn-v0_8-whitegrid', 'tableau-colorblind10']
```









Matplotlib 그래프
- 막대 그래프

- **막대그래프(Bar, Barh)**
  - 입력 데이터의 값을 막대모양의 길이로 나타낸 그래프

함수	입력값		사용방법
bar() barh()	필수	x, y 리스트: x, y 좌표 값	plt.bar(x_list, y_list)
	옵션	facecolor, color: bar 색상 edgecolor: bar 테두리 색상	plt.bar(x_list, y_list, color = "black", edgecolor = "red")
		alpha: 투명도 (0 ~ 1)	plt.bar(x_list, y_list, alpha = 0.7)
		width: bar 크기 linewidth: bar 테두리 굵기 - 기본 = 0.8	plt.bar(x_list, y_list, width = 1, linewidth = 3)

Matplotlib 그래프
- 막대 그래프(계속)

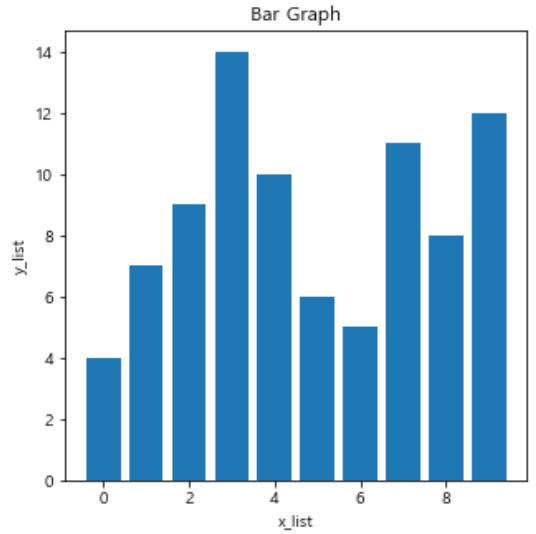
- 막대그래프(Bar, Barh) 예제

```
import matplotlib.pyplot as plt

x_list = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
y_list = [4, 7, 9, 14, 10, 6, 5, 11, 8, 12]

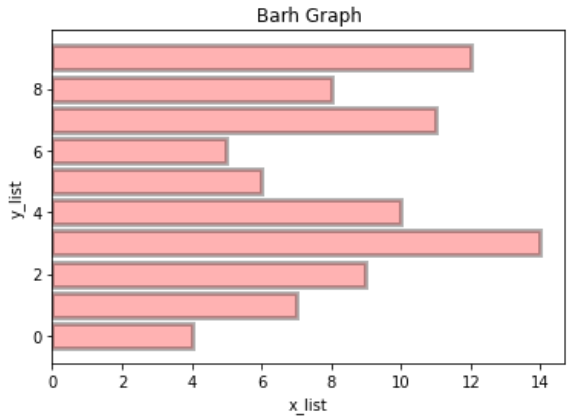
plt.title(" Bar Graph ")
plt.xlabel("x_list")
plt.ylabel("y_list")

plt.bar(x_list, y_list)
plt.show()
```




```
plt.title(" Barh Graph ")
plt.xlabel("y_list")
plt.ylabel("x_list")

plt.barh(x_list, y_list,
         facecolor = "red",          # 색
         edgecolor = "black",
         alpha = 0.3,                # 투명도
         linewidth = 3)              # bar 굵기
plt.show()
```



Matplotlib 그래프
- 산포도

- **산포도(Scatter)**
  - 입력 데이터가 얼마나 어떻게 퍼져 있는지 보여주는 지표

함수	입력값		사용방법
scatter()	필수	x, y 리스트: x, y 좌표 값	plt.scatter(x_list, y_list)
	옵션	facecolor, color, c: 점 색상 edgecolor: 점 테두리 색상	plt.scatter(x_list, y_list, c = "black", edgecolor = "red")
		s: 점 크기	plt.scatter(x_list, y_list, s =200)
		alpha: 투명도 (0 ~ 1)	plt.scatter(x_list, y_list, alpha =0.7)
		marker: 점의 모양	plt.scatter(x_list, y_list, marker ='v')
		linewidth: 점 테두리 굵기 - 기본 = 0.8	plt.scatter(x_list, y_list, linewidth = 3)

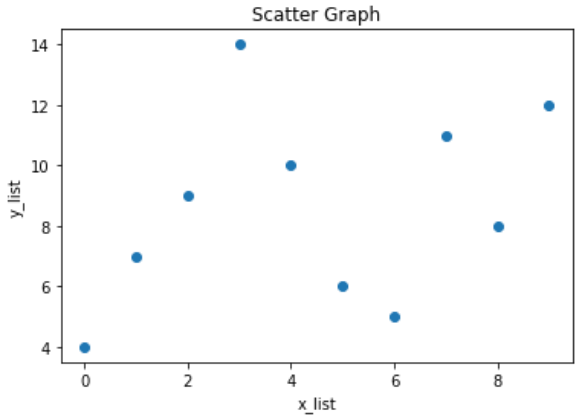
Matplotlib 그래프
- 산포도(계속)

- 산포도(Scatter) 예제

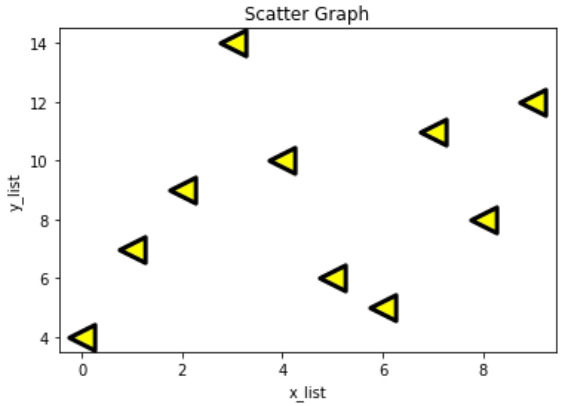
```
import matplotlib.pyplot as plt

plt.title(" Scatter Graph ")
plt.xlabel("x_list")
plt.ylabel("y_list")

plt.scatter(x_list, y_list)
plt.show()
```




```
plt.scatter(x_list, y_list,
            c = "yellow",                 # 색
            edgecolor = "black",
            s = 300,                      # 사이즈
            linewidth = 3,
            marker = '<')                 # 모양
plt.show()
```



Matplotlib 그래프
- 파이 차트

- **파이 차트(Pie Chart)**
  - 입력값의 합을 100으로 하고 각 데이터가 차지하는 비율을 원으로 표현

함수	입력값		사용방법
pie()	필수	리스트: 비율 데이터	plt.pie(x_list)
	옵션	labels: 범주 리스트	plt.pie(x_list, labels = y_list)
		stratangle: 시작 각도	plt.pie(x_list, startangle = 90)
		autopct: 비율값 표시 - 문자열 포멧 사용 ('%.2f')	plt.pie(x_list, autopct = '%.2f')
		counterclock: 출력순서 - 기본값: True(반시계반향)	plt.pie(x_list, counterclock = False)

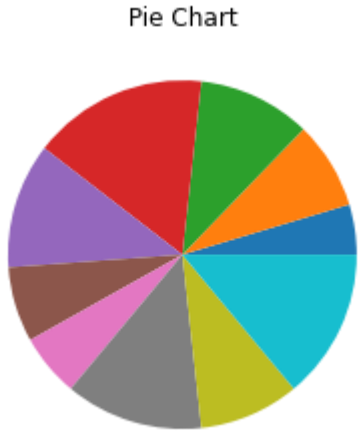
Matplotlib 그래프
- 파이 차트(계속)

- 파이 차트(Pie Chart) 예제

```
import matplotlib.pyplot as plt

plt.title(" Pie Chart ")

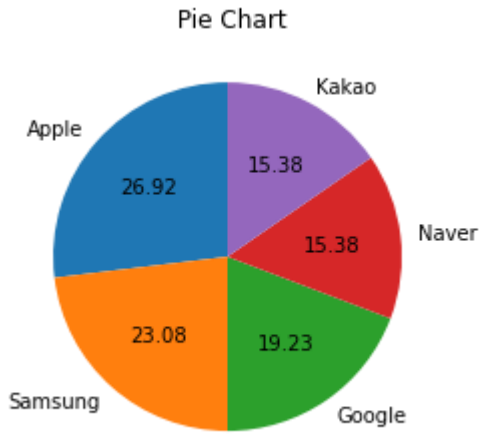
plt.pie(y_list)
plt.show()
```




```
ratio = [35, 30, 25, 20, 20]
labels = ['Apple', 'Samsung', 'Google', 'Naver', 'Kakao']

plt.title(" Pie Chart")

plt.pie(ratio,
        labels = labels,          # 레이블
        startangle = 90,          # 시작 위치
        autopct = '%.2f')         # 비율값 표시
plt.show()
```



Matplotlib 그래프
- 히스토그램

- **히스토그램(Histogram)**
  - 입력 값(전체 집단)의 분포 상태를 파악하거나 비교할 때 사용

함수	입력값		사용방법
hist()	필수	리스트: 입력 데이터	plt.hist(x_list)
	옵션	bins: 막대의 갯수(분포의 표현 너비)를 설정	plt.hist(x_list, bins = 7)
		density: 학률밀도 설정을 위해 데이터 정규화	plt.hist(x_list, density = True)
		histtype: 그래프 타입 - 'bar', 'barstacked', 'step', 'stepfilled'	plt.hist(x_list, histtype = 'step')
		alpha: 투명도(0 ~ 1)	plt.hist(x_list, alpha = 0.6)
		facecolor, color: bar 색상 edgecolor: bar 테두리 색상	plt.hist(x_list, facecolor = "black", edgecolor = "red")

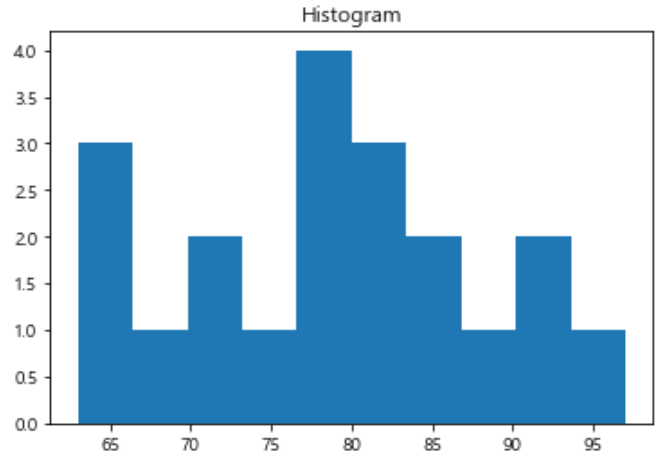
Matplotlib 그래프
- 히스토그램(계속)

- 히스토그램(Histogram) 예제

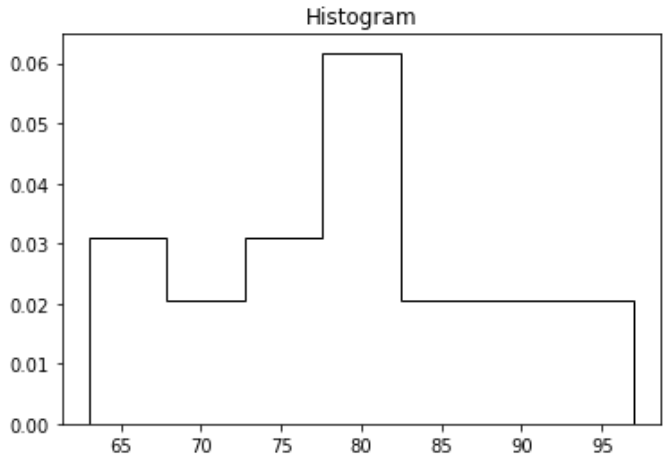
```
import matplotlib.pyplot as plt

hist_list =[93, 79, 80, 84, 80, 65, 73, 63, 72, 80, 68, 97, 79,
            74, 90, 86, 63, 78, 77, 92]
plt.title( "Histogram" )

plt.hist(hist_list)
plt.show()
```




```
plt.hist(hist_list,
         bins = 7,                  # 막대 너비
         density = True,            # 확률밀도 적용
         histtype = "step",         # 속이 빈 막대
         edgecolor = "black")       # 막대 테두리 색상
plt.show()
```



Pandas, Numpy에서 Matplotlib 사용
- Series Matplotlib

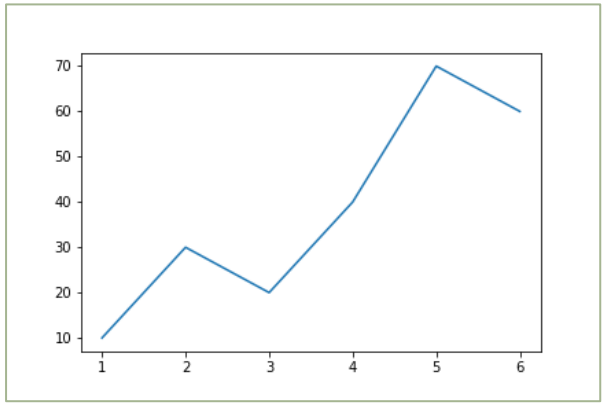
- 입력값: Series로 생성된 배열

```
from matplotlib import pyplot as plt
import pandas as pd

month = ["1", "2", "3", "4", "5", "6"]
block_chain = [10, 30, 20, 40, 70, 60]

plt.plot(month , block_chain)            # plot  생성
plt.savefig("My_block_chain1.png")   # 이미지로 저장
plt.show()
```

My_block_chain1.png




```
block_chain={"1" : 10, "2" : 30, "3": 20, "4" : 40, "5" : 70 ,"6" : 60}

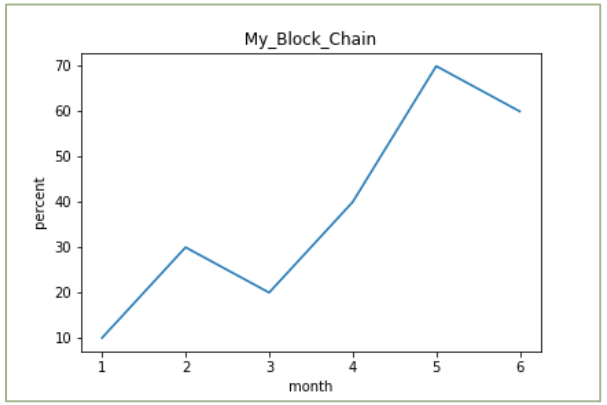
bc = pd.Series( block_chain )        # pd 스리즈 생성

plt.plot(bc)                                 # plot 생성

plt.xlabel( "month" )                  # x축 이름
plt.ylabel( "percent" )                # y축 이름
plt.title( "My_Block_Chain")       # 그래프 제목

plt.savefig( "My_block_chain2.png" )    # 이미지로 저장
plt.show()
```

My_block_chain2.png



Pandas, Numpy에서 Matplotlib 사용
- DataDrame Matplotlib

- **Pandas에서는 DataFrame을 위한 plot()함수를 제공**
  - subplots(): 여러개의 데이터가 있을 경우 그래프를 나누어서 시각화

- **쥬피터노트북 환경에서는 show()함수 사용없이 출력 가능**

함수	기능	사용
plot() plot.line()	선으로 표기 옵션 - subplots=True	데이터프레임.plot() 데이터프레임.plot.line() 데이터프레임.plot.line(subplots = True)
plot.bar()	막대그래프 옵션 - subplots=True	데이터프레임.plot.bar() 데이터프레임.plot(kind = "bar") 데이터프레임.plot(kind = "bar", subplots = True)
plot.box()	box 그래프 옵션 - subplots=True	데이터프레임.plot.box() 데이터프레임.plot(kind = "box") 데이터프레임.plot(kind = "box", subplots = True)
plot.pie()	pie 차트 옵션 - subplots=True - y = "name"	데이터프레임.plot.pie() 데이터프레임.plot(kind = "pie", subplots=True) 데이터프레임.plot(kind = "pie", y = "행 이름")

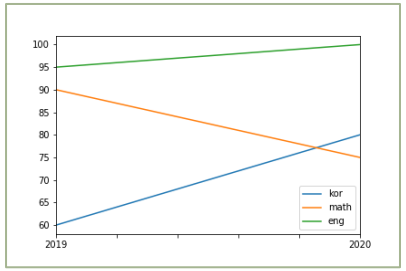
Pandas, Numpy에서 Matplotlib 사용
- DataDrame Matplotlib

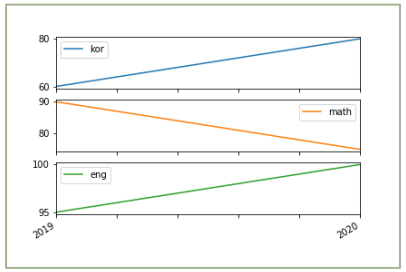
```
my_score = [[60, 90, 95], [80, 75, 100]]
subject = ["kor", "math", "eng"]
year = ["2019", "2020"]

df = pd.DataFrame(my_score, columns = subject, index = year)
```

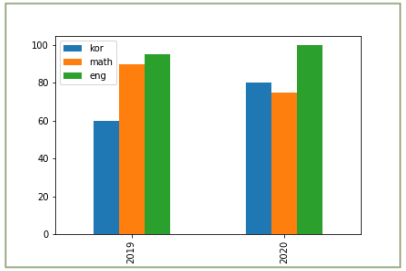
	kor	math	eng
2019	60	90	95
2020	80	75	100

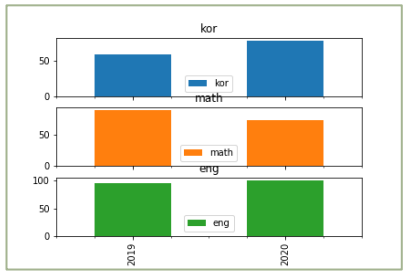
```
df.plot()
df.plot(subplots=True)
```



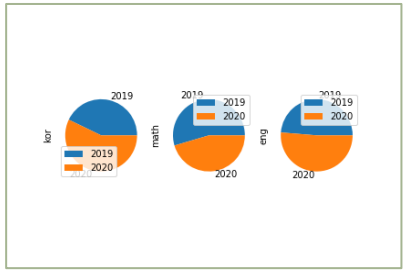


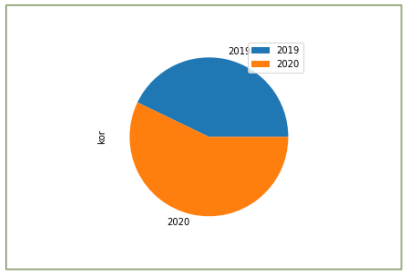

```
df.plot(kind = "bar")
df.plot.bar(subplots=True)
```






```
df.plot(kind = "pie", subplots=True)
df.plot(kind = "pie", y = "kor")
```





Pandas, Numpy에서 Matplotlib 사용
- numpy Matplotlib

- 입력값: Numpy로 생성된 배열

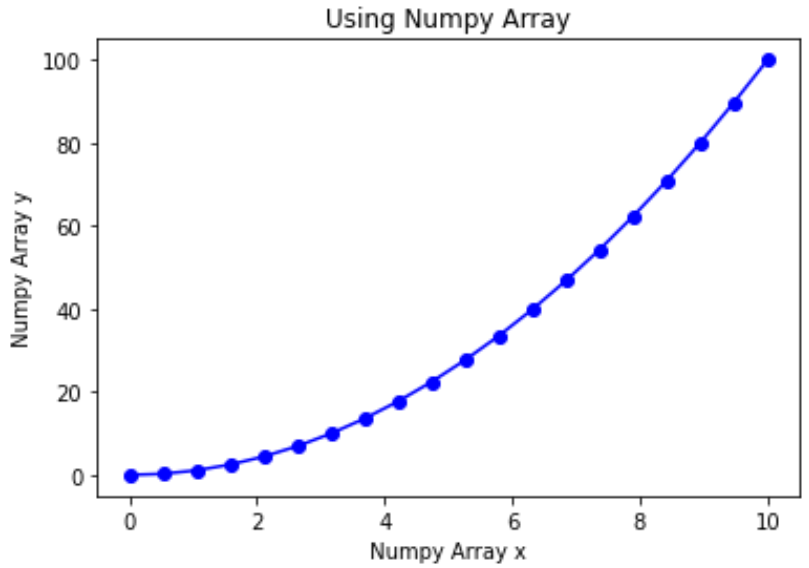
```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 20)    #넘파이 배열 x 생성

y = x**2.0                    #넘파이 배열 y 생성

plt.title("Using Numpy Array")
plt.xlabel(" Numpy Array x ")
plt.ylabel(" Numpy Array y ")

plt.plot( x, y, "-ob" )
plt.show()
```



np.linspace(0 , 10, 20) : 0부터 10 사이까지 20개의 선형적인 원소 값들을 배열 생성

실습

- **공공데이터 수집 및 활용**
  - 수집된 데이터를 정제하여 원하는 값을 Matplotlib으로 출력

- **기상자료개방포털**
  - 지상, 해양, 고층, 항공관측, 위성, 레이더, 수치예보모델자료 등
    총 30종류의 날씨데이터
  - 우리나라 100년 이상의 기후통계 제공
    - 기온, 강수량, 극값순위, 기후평년값, 장마, 황사, 폭염, 열대야 일수 등



https://data.kma.go.kr

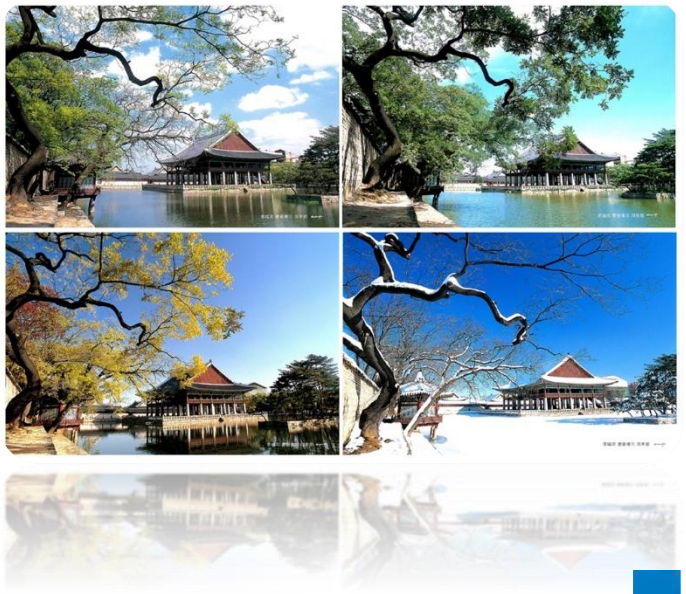
실습 데이터

- **기상자료개방포털 홈페이지**
  - [기후통계분석] - [조건별통계] – [지역,기간 설정] – [검색]
  - 기간: 일(2000~2026)



실습 데이터

- **서울지역 기온의 시계열 정보**
  - 기후통계분석 – 통계분석 – 기온분석
  - 날짜(일,월,년)의 평균, 최저, 최고기온 조회
  - 기간: 2000/01/01 ~ 2026/08/31
  - 지역: 서울(108)
  - 파일명: 'seoul.csv'



seoul.csv

	A	B	C	D	E
1	날짜	지점	평균기온(℃)	최고기온(℃)	최저기온(℃)
2	2000-01-01	108	5.5	9.9	1.8
3	2000-01-02	108	4.2	6.9	-0.9
4	2000-01-03	108	-2.2	0.1	-4.6
5	2000-01-04	108	0.3	4.3	-4.3
6	2000-01-05	108	2.8	4.6	0.1
7	2000-01-06	108	1.7	5.7	-4.2
8	2000-01-07	108	-8.2	-4.2	-12.1
9	2000-01-08	108	-3.8	-0.1	-7.2
10	2000-01-09	108	-1.9	1	-6
11	2000-01-10	108	-0.8	2.3	-4.8
12	2000-01-11	108	-0.2	4.9	-7.1
13	2000-01-12	108	3.7	5.1	2.6
14	2000-01-13	108	0.7	4	-2.1
15	2000-01-14	108	-1.4	1.1	-2.6
16	2000-01-15	108	-1.1	3.3	-4.9
17	2000-01-16	108	0.2	4.5	-2.2
18	2000-01-17	108	-0.9	3.8	-4.4
19	2000-01-18	108	-1.8	2.3	-5.1
20	2000-01-19	108	-4.7	-1.2	-7.5
21	2000-01-20	108	-8.5	-5.4	-11.4
22	2000-01-21	108	-7	-2.6	-11.6
23	2000-01-22	108	-2.5	1.6	-6.7
24	2000-01-23	108	2.4	7.4	-1.1
25	2000-01-24	108	-2.9	0.2	-6.5

실습 데이터 열기

- CSV 파일 열고 읽기

```
import csv                          # csv 파일을 다루기 위한 패키지

f = open('seoul.csv')               # open() 함수: 파일 열기
                                    # 입력값: 주소가 포함된 파일명

data = csv.reader( f )              # csv.reader() 함수:
                                    # 파일의 내용을 읽을 수 있게 data에 저장

for row in data:                    # 반복문 for에 의해서 data의 내용을
    print( row )                    # 한줄씩 읽어 반복변수 row에 리스트 저장
```

row

```
['날짜', '지점', '평균기온(℃)', '최저기온(℃)', '최고기온(℃)']
['1907-10-01', '108', '13.5', '7.9', '20.7']
['1907-10-02', '108', '16.2', '7.9', '22']
['1907-10-03', '108', '16.2', '13.1', '21.3']
['1907-10-04', '108', '16.5', '11.2', '22']
['1907-10-05', '108', '17.6', '10.9', '25.4']
['1907-10-06', '108', '13', '11.2', '21.3']
['1907-10-07', '108', '11.3', '6.3', '16.1']
['1907-10-08', '108', '8.9', '3.9', '14.9']
['1907-10-09', '108', '11.6', '3.8', '21.1']
['1907-10-10', '108', '14.2', '6.4', '24.1']
['1907-10-11', '108', '15.4', '10.1', '20.4']
['1907-10-12', '108', '13.9', '11.1', '17.4']
```

index: 0 1 2 3 4

row(list): ['1907-10-01', '108', '13.5', '7.9', '20.7']

row[0]: ' 1907-10-01 ' 날짜(문자열)
row[1]: ' 108 ' 지점(문자열)
row[2]: ' 13.5 ' 평균기온(문자열)
row[3]: ' 7.9 ' 최저기온(문자열)
row[4]: ' 20.7 ' 최고기온(문자열)

실습 데이터 다루기

- 최고기온 저장

```
import csv

f = open('seoul.csv')
data = csv.reader( f )

result = []     # 최고 기온 값을 저장할 리스트 생성

next(data)  # next()함수: 현재 반복에 사용될 값(첫 줄 )을 반환
            # 다음 반복문에는 다음 값(다음 줄) 부터 반복에 사용

for row in data:
    if row[ -1 ] != '' :    # 비어 있지 않을 경우(값이 있을 경우)
        result.append( float( row[-1] ) )

print(result)
```

문자열을 실수로 변환

result 리스트에 추가

row[-1]

	A	B	C	D	E
1	날짜	지점	평균기온(℃)	최저기온(℃)	최고기온(℃)
2	1907-10-01	108	13.5	7.9	20.7
3	1907-10-02	108	16.2	7.9	22
4	1907-10-03	108	16.2	13.1	21.3
5	1907-10-04	108	16.5	11.2	22
6	1907-10-05	108	17.6	10.9	25.4
7	1907-10-06	108	13	11.2	21.3
8	1907-10-07	108	11.3	6.3	16.1
9	1907-10-08	108	8.9	3.9	14.9
10	1907-10-09	108	11.6	3.8	21.1
11	1907-10-10	108	14.2	6.4	24.1
12	1907-10-11	108	15.4	10.1	20.4
13	1907-10-12	108	13.9	11.1	17.4
14	1907-10-13	108	13.8	8.3	21.3
15	1907-10-14	108	13	6.1	20.6
16	1907-10-15	108	13.1	5.7	20.9
17	1907-10-16	108	14.1	8.2	20.2

출력결과

```
[20.7, 22.0, 21.3, 22.0, 25.4, 21.3, 16.1, 14.9, 21.1, 24.1, 20.4, 17.4, 21.3, 20.6, 20.9, 20.2, 21.6, 20.9,
21.3, 22.7, 19.9, 19.6, 16.3, 17.1, 18.7, 18.2, 20.7, 19.6, 20.0, 20.1, 20.3, 21.3, 21.1, 11.1, 13.6, 17.0, 1
8.1, 12.4, 9.4, 11.9, 13.2, 13.4, 14.9, 16.4, 15.2, 17.6, 12.6, 11.7, 6.7, 4.3, 7.1, 6.2, 11.2, 9.9, 5.9, -0.
7, 1.5, 1.9, 2.0, 2.9, -2.4, 1.4, 3.4, 6.1, 1.3, 2.6, 5.8, 1.9, 6.6, 8.1, 4.3, 0.6, 3.8, 9.0, 6.4, -5.1, -2.
4, -2.1, -0.4, -0.2, 1.9, -8.1, -10.2, -9.4, 0.9, 6.8, 6.4, -0.7, -1.9, -4.4, -6.1, -2.1, -1.3, -3.0, -2.9, -
1.7, -0.1, 2.9, -1.2, -0.9, 2.0, 3.3, 4.8, 4.5, 4.7, 0.1, 0.9, -4.5, -3.6, 2.8, 2.8, 2.3, -6.1, -5.1, -2.7,
0.1, 0.3, -0.6, 0.0, -1.3, 0.8, 0.8, 6.2, 1.6, -0.2, -5.4, -4.1, -5.5, -5.7, -3.7, 1.3, -4.1, -2.5, -2.8, -1.
2, 1.3, 2.3, 2.0, -1.4, -1.6, 0.3, 5.5, 7.3, 5.1, 5.4, 2.7, 1.7, 0.8, 4.1, 8.4, 6.3, 5.6, 4.0, 0.1, 4.9, 4.0,
7.9, 7.9, 2.2, 1.9, 1.5, 1.5, 1.8, 2.6, 3.8, 6.7, 11.8, 13.5, 15.7, 10.7, 7.9, 4.3, 6.0, 12.5, 17.0, 14.7, 1
1.4, 7.4, 8.0, 15.5, 18.2, 11.4, 13.2, 14.8, 14.9, 11.1, 15.0, 9.9, 10.6, 13.6, 17.2, 16.9, 17.1, 13.6, 9.4,
```

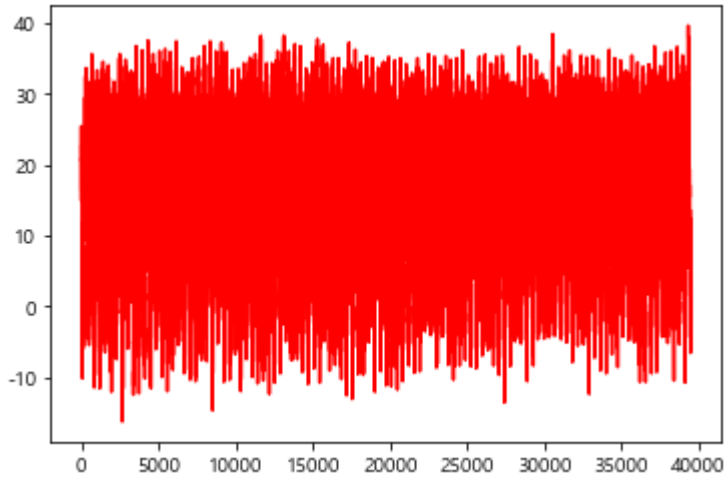
실습 데이터 다루기(계속)

- 최고기온 matplotlib으로 출력

```
import csv
import matplotlib.pyplot as plt     # matplotlib 패키지 가져오기

f = open('seoul.csv')
data = csv.reader( f )
result = []
next(data)
for row in data:
    if row[ -1 ] != '' :
        result.append( float( row[-1] ) )

plt.plot( result, 'r' )  # result에 저장된 최고기온 입력
plt.show()
```



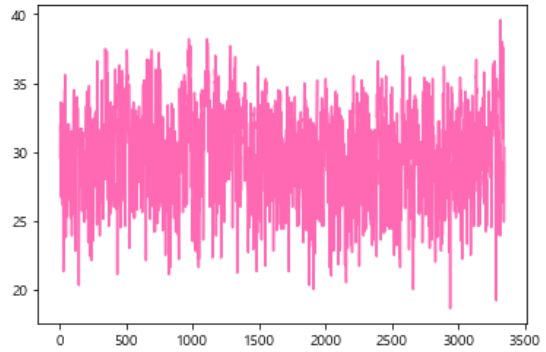
실습 데이터 다루기(계속)

- 특정 월의 기온 저장

```
import csv
import matplotlib.pyplot as plt

f = open('seoul.csv')
data = csv.reader( f )
result = []
next(data)
for row in data:
    if row[ -1 ] != '' :
        if row[0].split( '-' )[1] == '08' :    # 8월일 경우
            result.append( float( row[-1] ) )

plt.plot( result, 'hotpink' )
plt.show()
```



row[0]

```
['1907-10-01', '108', '13.5', '7.9', '20.7']
```

row[0].split(' – ')

```
1907-10-01
```

```
0      1     2
['1907', '10', '01']
```

```
row[0].split( ' – ' )[0]      # '1907'
row[0].split( ' – ' )[1]      # '10'
row[0].split( ' – ' )[2]      # '01'
```

실습 데이터 다루기
- numpy, pandas

```
import csv

dates = []
avg_temps = []
max_temps = []
min_temps = []

f = open('seoul.csv', 'r')  # with 없이 그냥 open
reader = csv.reader(f)
header = next(reader)  # 헤더 줄 건너뛰기

for row in reader:
    if row[ -1 ] != '' :
        dates.append(row[0])                    #날자 데이터
        avg_temps.append(float(row[2]))         # 평균기온
        max_temps.append(float(row[3]))         # 최고기온
        min_temps.append(float(row[4]))         # 최저기온

f.close()  # with문이 없으니 반드시 직접 닫아줘야 함

print(dates[:3])
print(avg_temps[:3])
print(len(dates))
```

```
['2000-01-01', '2000-01-02', '2000-01-03']
[5.5, 4.2, -2.2]
9752
```

실습 데이터 다루기
- numpy, pandas(계속)

- **Numpy 적용 및 활용**

```python
import numpy as np

avg_arr = np.array(avg_temps)
max_arr = np.array(max_temps)
min_arr = np.array(min_temps)

print(f"평균: {np.mean(avg_arr):.2f}°C")
print(f"표준편차: {np.std(avg_arr):.2f}")
print(f"최고: {np.max(max_arr)}°C")
print(f"최저: {np.min(min_arr)}°C")

print()

diff_arr = max_arr - min_arr

print(f"평균 일교차: {np.mean(diff_arr):.2f}°C")
print(f"일교차가 가장 큰 날: {dates[np.argmax(diff_arr)]} ({np.max(diff_arr)}°C)")
print(f"일교차가 가장 작은 날: {dates[np.argmin(diff_arr)]} ({np.min(diff_arr)}°C)")
```

```
평균: 13.21°C
표준편차: 10.57
최고: 39.6°C
최저: -18.6°C

평균 일교차: 8.49°C
일교차가 가장 큰 날: 2025-04-12 (21.1°C)
일교차가 가장 작은 날: 2024-07-29 (1.0°C)
```

실습 데이터 다루기
- numpy, pandas(계속)

- **Numpy 적용 및 활용**
  - 33도 이상의 폭염일 찾기

```
hot_mask = max_arr >= 33
hot_days = np.array(dates)[hot_mask]
hot_temps = max_arr[hot_mask]

print(f"폭염일수: {hot_mask.sum()}일")
for d, t in zip(hot_days, hot_temps):
    print(f"{d}: {t}°C")
```

```
폭염일수: 312일
2000-06-18: 33.9°C
2000-06-19: 33.3°C
2000-06-28: 33.4°C
2000-07-02: 33.0°C
2000-07-03: 33.2°C
2000-07-04: 33.8°C
2000-07-05: 33.9°C
2000-07-06: 34.2°C
2000-07-07: 33.4°C
2000-07-08: 33.0°C
2000-07-10: 34.0°C
2000-07-27: 33.1°C
2000-08-02: 35.1°C
```

실습 데이터 다루기
- numpy, pandas(계속)

- **pandas DataFrame 생성**

```
import pandas as pd

df = pd.DataFrame({
    '날짜': pd.to_datetime(dates),
    '평균기온': avg_temps,
    '최고기온': max_temps,
    '최저기온': min_temps
})

df.set_index('날짜', inplace=True)
df['일교차'] = df['최고기온'] - df['최저기온']

df.head()
```

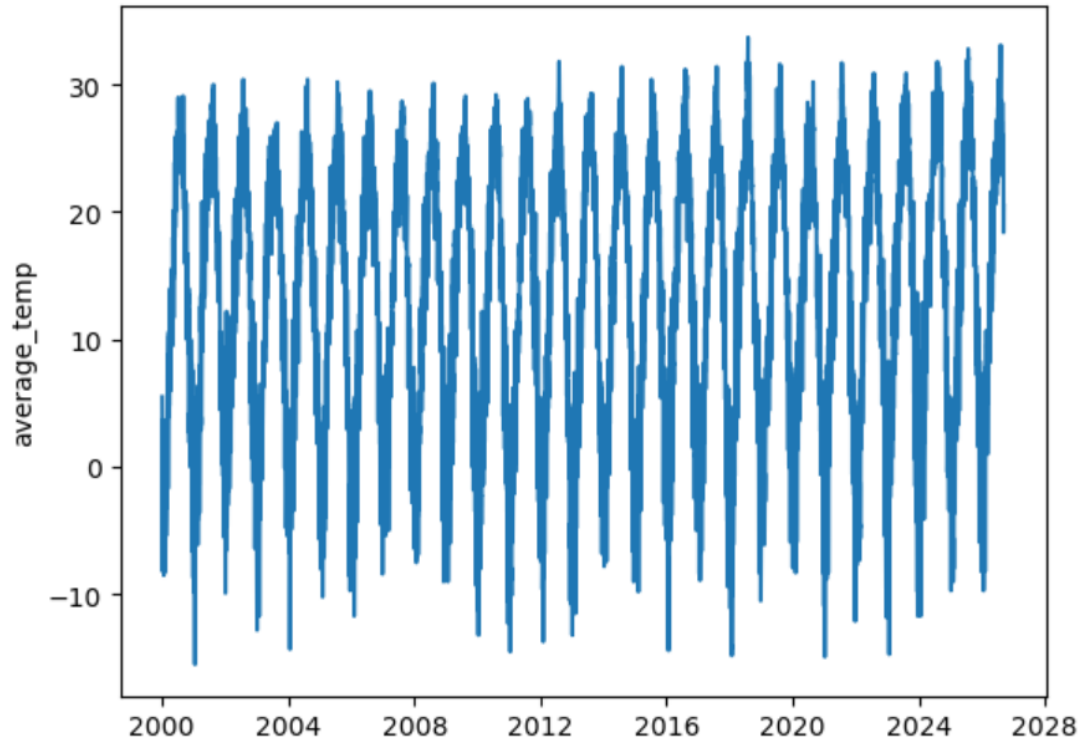
날짜	평균기온	최고기온	최저기온	일교차
2000-01-01	5.5	9.9	1.8	8.1
2000-01-02	4.2	6.9	-0.9	7.8
2000-01-03	-2.2	0.1	-4.6	4.7
2000-01-04	0.3	4.3	-4.3	8.6
2000-01-05	2.8	4.6	0.1	4.5

실습 데이터 다루기
- numpy, pandas(계속)

- **matplotlib 시각화**

```
import matplotlib.pyplot as plt

plt.plot(df.index, df['평균기온'])
plt.xlabel("year")
plt.ylabel("average_temp")
plt.show()
```



요약

- **데이터를 시각화하기 위한 파이썬의 대표적인 라이브러리**

- **기본 함수에 옵션을 주어서 다양한 시각화 기능을 제공**

- **Matplotlib 그래프**
  - 막대그래프(Bar, Barh)
  - 산포도(Scatter)
  - 파이 차트(Pie Chart)
  - 히스토그램(Histogram)

- **Pandas나 Numpy로 생성된 배열 또한 Matplotlib의 입력값으로 넣어 시각화 가능**



강남대학교

KANGNAM UNIVERSITY